

49. Group Anagrams

Medium · Array · Hash Table · String · Sorting

Approach — Frequency Signature as Hash Key

Two strings are anagrams of each other if and only if they contain exactly the same characters with the same frequencies. So instead of sorting each string to get a canonical form (which costs $O(L \log L)$ per string), this solution builds a **frequency signature** — a count of each of the 26 lowercase letters — and uses that signature as a hash map key.

For each string `s`, a 26-element count array `cnt` is built where `cnt[c - 'a']` holds the frequency of character `c`. This array is then converted into a unique string key by concatenating each count separated by '#', e.g. `"#1#0#2..."`. All strings producing the same key are anagrams of each other and get grouped together in the hash map.

Key Insight: The `"#"` separator prevents ambiguity between counts like 1 and 12 versus 11 and 2 — without it, different frequency distributions could accidentally produce the same concatenated string. Building this signature takes only $O(L)$ per string, faster than sorting.

Step-by-step Breakdown

Step 1 — Initialize result and hash map

Create `ans` to hold the final grouped result, and `hashMap` — an `unordered_map` — mapping each frequency signature to the list of strings sharing it.

Step 2 — Count character frequencies

For each string `s` in `strs`, build a 26-element array `cnt` where `cnt[c - 'a']` is incremented for every character `c` in `s`. This array captures the exact letter composition of `s`, regardless of order.

Step 3 — Build the signature key

Convert `cnt` into a string key by appending `"#" + to_string(n)` for each count `n`. This produces a unique, order-independent fingerprint for the string's character composition. Append the original string `s` to `hashMap[key]`.

Step 4 — Collect groups into the answer

After processing all strings, iterate over `hashMap`. Each value `v` is a complete group of anagrams — move it directly into `ans` using `move(v)` to avoid copying, then return `ans`.

Solution Code

```
1  class Solution {
2  public:
3      vector<vector<string>> groupAnagrams(vector<string>& strs) {
4          vector<vector<string>> ans;
5          unordered_map<string, vector<string>> hashMap;
6
7          for(auto &s: strs)
8          {
9              vector<int> cnt(26);
10             for(auto &c: s) cnt[c-'a']++;
11
12             string key = "";
13             for(auto &n: cnt) key += "#" + to_string(n);
14
15             hashMap[key].push_back(s);
16         }
17         for(auto &[u, v]: hashMap)
18         {
19             ans.push_back(move(v));
20         }
21         return ans;
22     }
23 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(N \times L)$	$O(N \times L)$

For each of N strings, building the 26-count array and the signature key both take $O(L)$, where L is the string's length. Total: $O(N \times L)$, avoiding the $O(N \times L \log L)$ cost of sorting each string.

The hash map stores every input string once (across all groups), plus a 26-character-derived key per string — overall $O(N \times L)$ space.